

A two-week Glassbox pilot

For a team lead or IT owner who wants to know what Claude Code is doing on their developers' machines before deciding anything. Five people, two weeks, nothing installed that runs on its own, nothing that leaves the building. At the end: one report and a 30-minute readout.

What you get

One fleet report (`glassbox collect`) over every Claude Code session the five ran in the fortnight: how much it cost and where it concentrated, which of the twenty-odd rules fired and how often, one line per developer, and the files that were read too many times — as keys, not names. Plus a readout that turns the numbers into three or four habits worth changing, and a view on whether the paid audit is worth it for the whole team. If the report shows nothing you didn't already know, that's the answer too.

What you don't get, and don't have to give

- No hooks. The `Stop` and `SessionStart` hooks and the `PreToolUse` loop guard (`docs/IT.md §6`) stay off; nothing runs unless a person runs it.
- No transcript leaves a machine. The only thing that moves is a JSON of counts, timings, rule ids and hashed file keys.
- No names in the report unless you want them: developers pick their own source name (`--out <share>/<name>.json`) and it can be `dev-3`.
- No install on the fleet. `npx glassbox-trace@<version>` fetches the package into npm's cache on first run, or IT puts one pinned copy on a share. Node 18+ is the only requirement.
- No change to how anyone uses Claude Code during the fortnight. The point is to see what normal looks like.

Who and what

Five developers who use Claude Code most days, ideally a mix (one who lives in it, one who is sceptical, one on a big refactor, one on small tickets, one who runs subagents or workflows). One shared folder they can write to (a file share, a Git repo they push to, a SharePoint library — anything). One owner who runs the collect at the end.

The fortnight

Day 0 — set up (15 minutes each). Each developer runs, once, on their own machine:

```
npx glassbox-trace@0.10.0 list --last 5
```

That proves Node and the package work and shows the sessions Glassbox can see (Claude Code writes them to `~/.claude/projects` by itself). Nothing else changes.

Optionally, the same day, each developer also runs the secrets sweep on their own machine and keeps the result to themselves:

```
npx glassbox-trace@0.10.0 fence
```

It lists any credential that has already reached a transcript on that machine — masked, with a fingerprint, never the value. If it finds one, the developer rotates that key and runs `fence --shred` to overwrite it in place. The pilot report never sees this output; the point is that nobody copies a report to the share on day 14 from a machine that still has a live key in a transcript.

Days 1-14 — nothing. Work as normal. Sessions accumulate on each machine as they always have.

Day 14 — each developer runs one command and copies one file to the share:

```
npx glassbox-trace@0.10.0 check --all --since 14d --redact --legend audit.legend.json --format json > <share>/<name>.json
```

On Windows PowerShell, run it through `cmd` so the file is saved as UTF-8 (0.9.1 and later also read the UTF-16 file PowerShell's own `>` writes, but older versions skip it):

```
cmd /c "npx glassbox-trace@0.10.0 check --all --since 14d --redact --legend audit.legend.json --format json > <share>\<name>.json"
```

`--redact` drops every string from the transcript (prompts, commands, results, titles); `--legend` turns file paths into `file:1a2b3c4d` keys and keeps the key→path map in `audit.legend.json` on that machine. The developer can open the JSON before copying it — it is meant to be read. Search it for a word only their code knows; it won't be there.

Day 15 — the owner runs collect on the folder:

```
npx glassbox-trace@0.10.0 collect <share> --format md --out fleet.md
```

and sends `fleet.md` to whoever is doing the readout. Unredacted files are skipped with a message, so a mistake on one machine cannot leak through the report.

Day 16 — readout, 30 minutes. Walk the report top to bottom: totals, the concentration line ("2 sessions carried half the spend"), the rules table with its "next time" column, the by-source table, the hammered files. For each of the top three findings, the developer whose machine it came from runs `glassbox reveal fleet.md --legend audit.legend.json` locally to see which file it was — the names stay in the room. Leave with three habits and a yes/no on going wider.

What the numbers mean

Costs are estimates from Anthropic's list prices unless the team passes its own rate card (`--rates`), so if you are on seats or credits treat the dollar column as a proportion, not an invoice; the token and time columns are exact. Findings are mechanical rules with published thresholds (README, "What it flags"), tuned so the median session has two of them and none at warn or above. A session with ten is unusual; a rule that fires in every session is a habit.

After the pilot

Three ways it goes: nothing worth changing (you're done, and it cost you an hour); a few habits worth fixing (the readout gives you the wording, and `glassbox hook install --feedback` lets the agent hear it itself, if you want that); or enough spend concentrated in enough sessions that a proper audit pays for itself — that conversation starts from your own numbers, not a pitch.

Questions: hi@aldo.ltd · Aldo Hushi, BlueprintAU, Adelaide.

